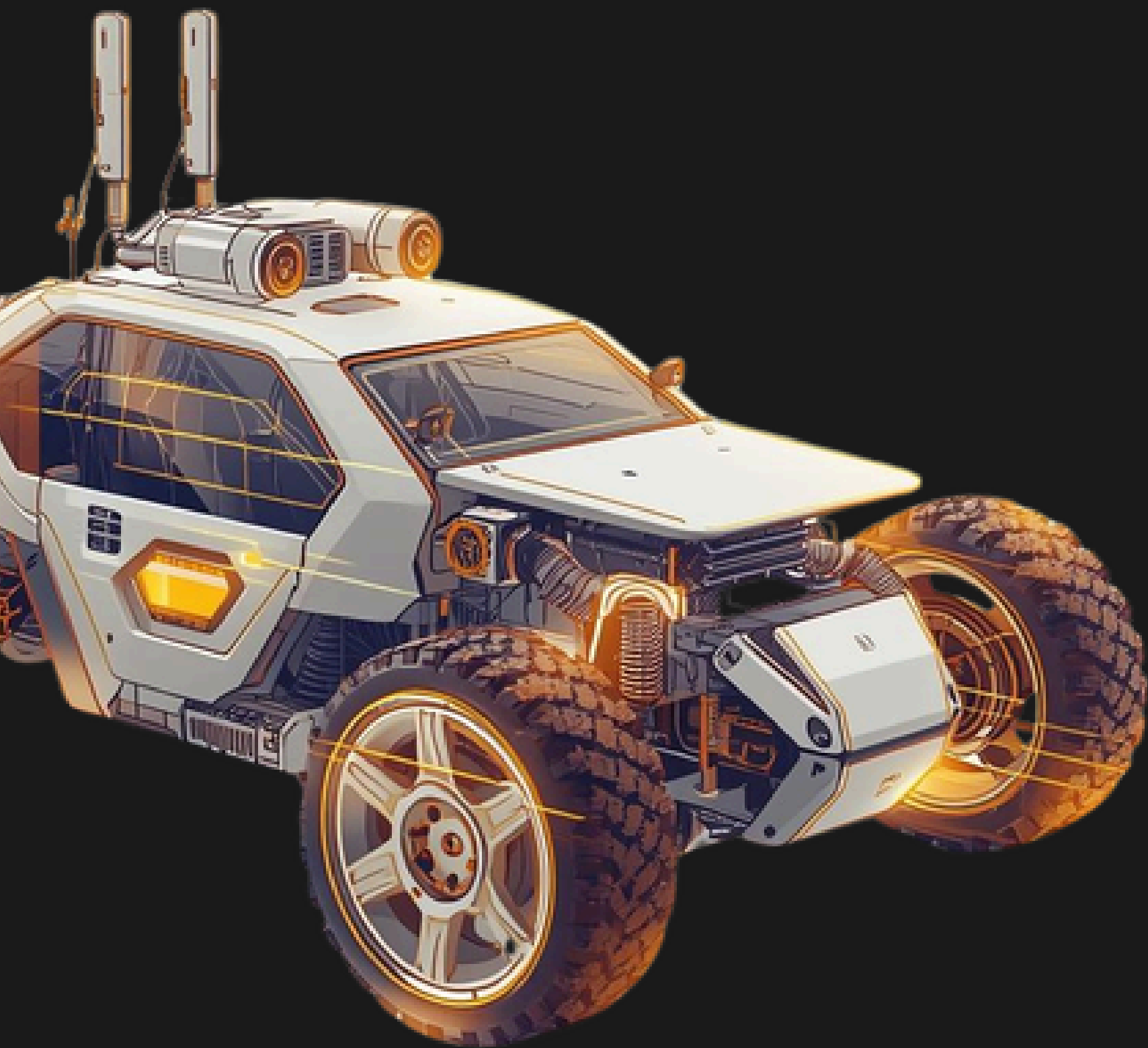




Line Follower Robot – PID Control System

PRESENTED BY
ADHAM ASHRAF
MOHAMED AMGAD
ZIAD YASSER



Agenda



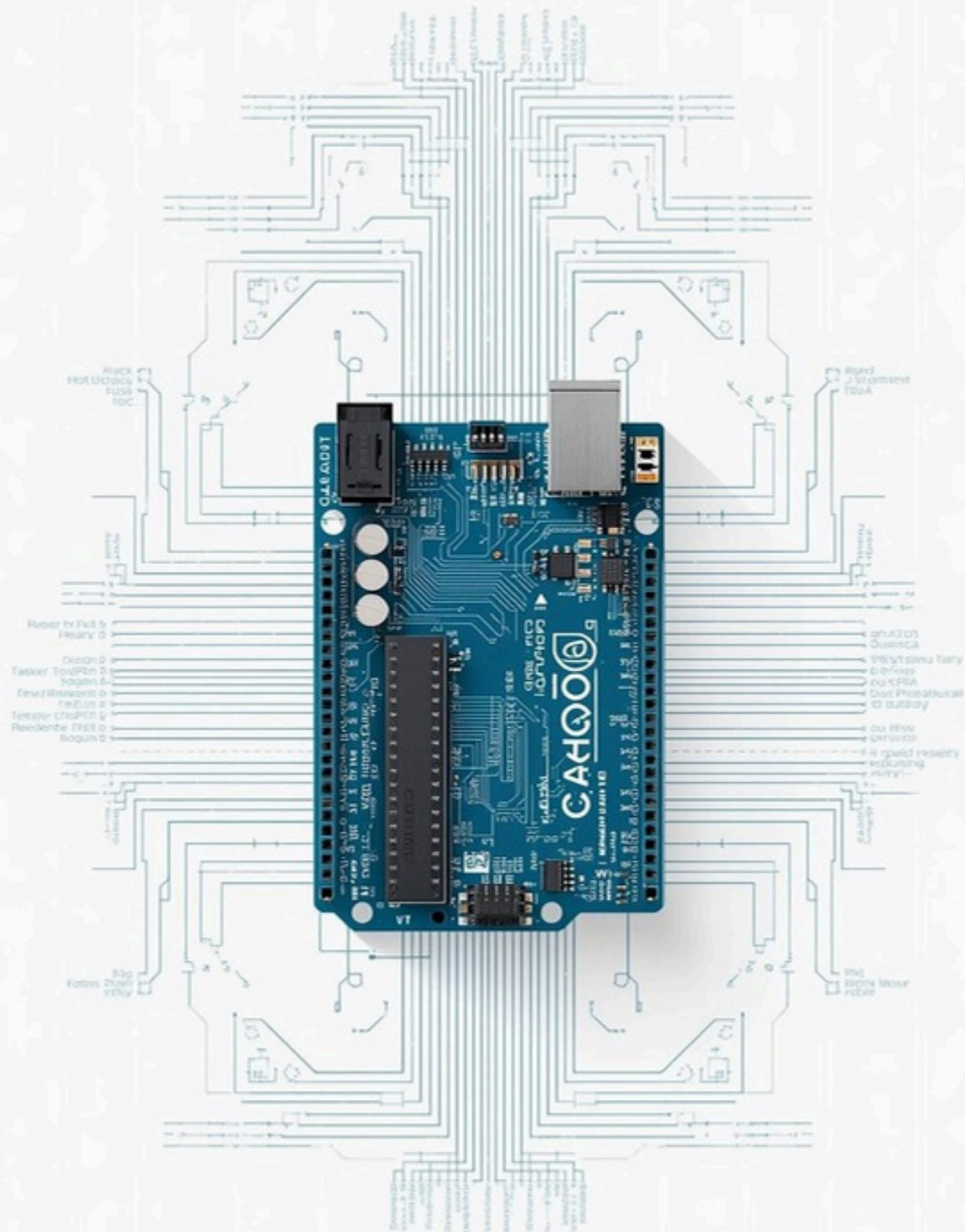
GOAL

Design, analyze, and implement a discrete PID control system for a line-following mobile robot.



KEY OBJECTIVES

- Control the steering angle based on line position error.
- Minimize the tracking error $e(t)$ to keep the robot centered on the line.
- Derive the system Transfer Function using System Identification.
- Implement the controller on Arduino hardware.

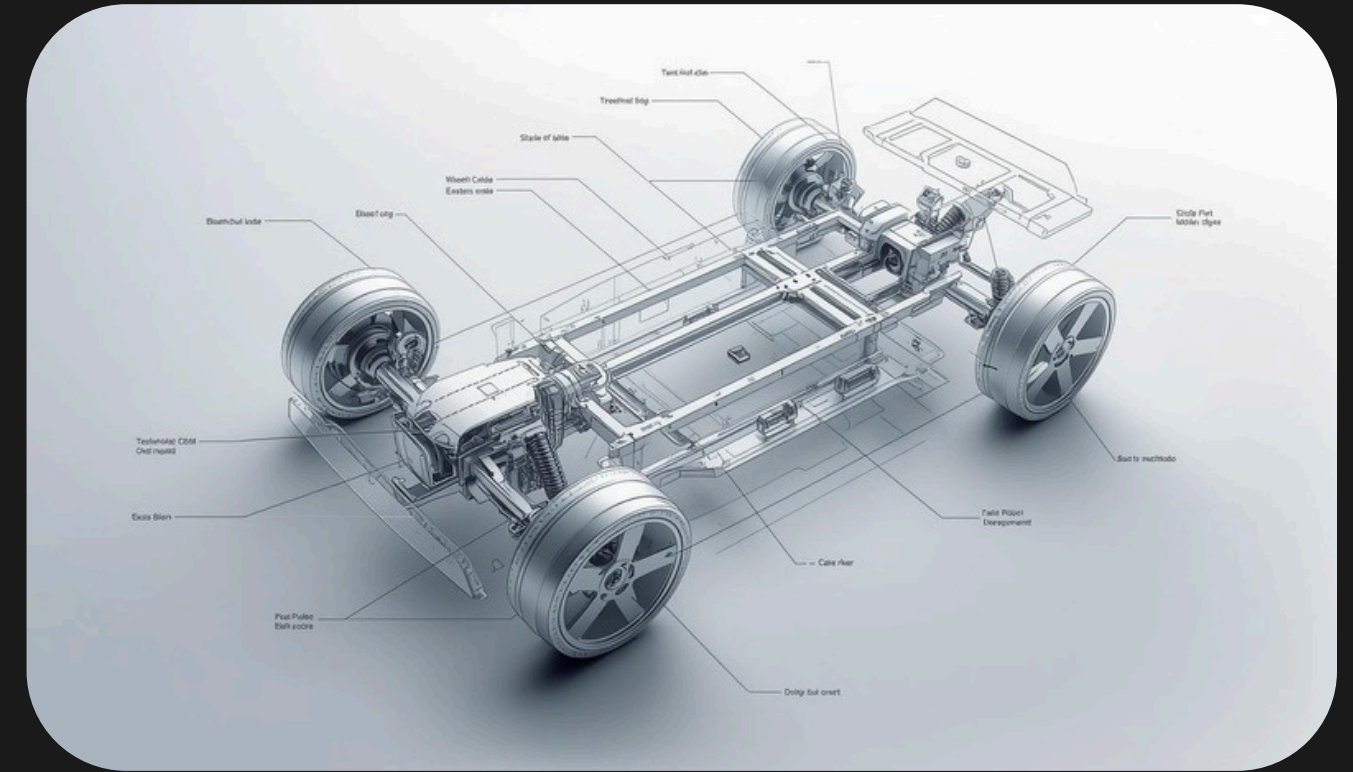


ARDUINO DESIGRAMITS

Transfer Function Derivation

* METHODOLOGY: SYSTEM IDENTIFICATION

- We applied step inputs to the motor speed differential.
- We measured the resulting lateral error to build an input-output dataset.



* ANALYSIS:

- The system is modeled as a second-order system.
- This function represents the relationship between the steering command (input) and the robot's lateral deviation (output).

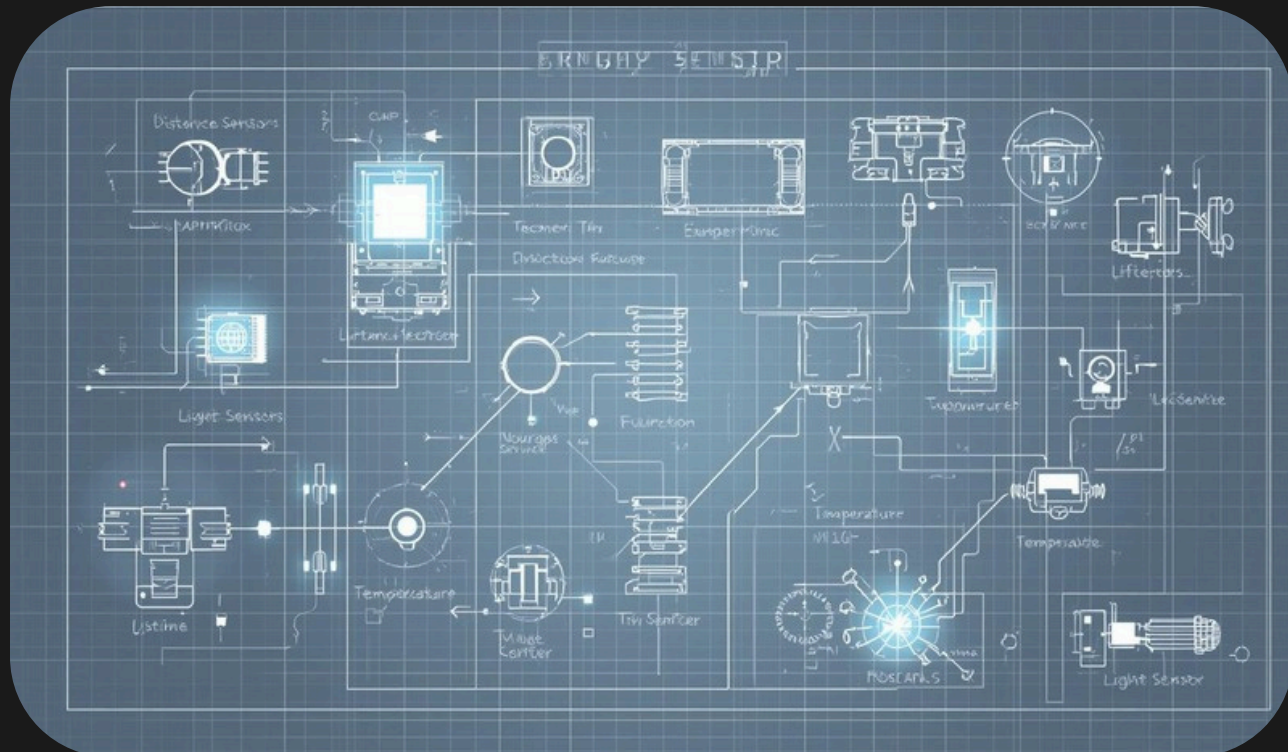
*** DERIVED PLANT MODEL $G(s)$:**

$$G(s) = \frac{0.3581}{s^2 + 4.657s + 2.845}$$

Controller Design & Discretization

* DISCRETIZATION:

Converted the Plant $G(s)$ to the Z-domain $G(z)$ to match the digital nature of the Arduino.



* PID TUNING:

- Used MATLAB PID Tuner GUI on the discrete plant model.
- Objective: Obtain optimal K_p , K_i , and K_d gains for fast response and low overshoot.

* CONTROLLER EQUATION:

Implemented in the Z-domain as:

$$C(z) = K_p + K_i z^{-1} + K_d(1 - z^{-1})$$

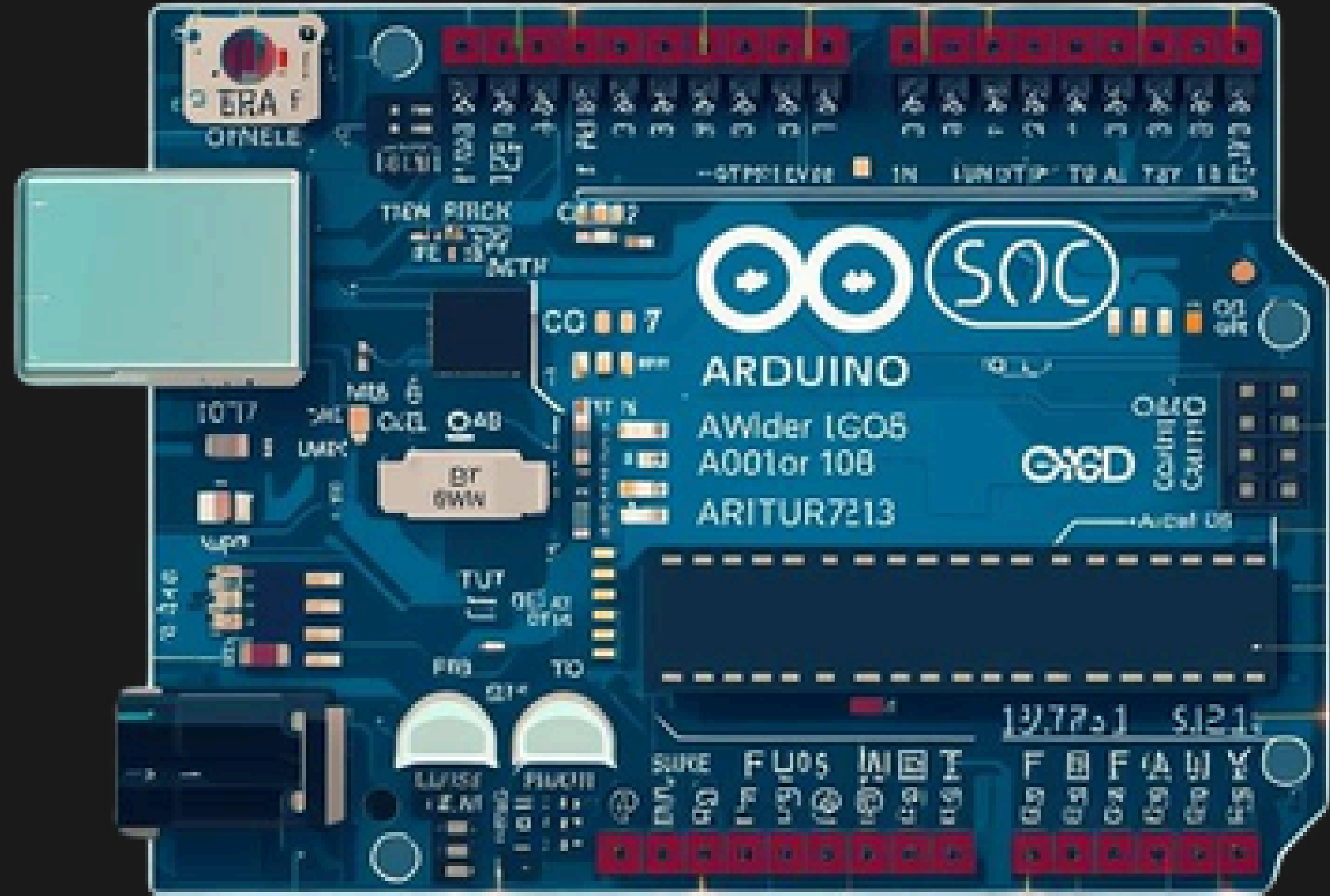
Stability Analysis

* TECHNIQUES USED:

- Pole-Zero Map: Checked the location of poles on the Z-plane.
- Stability Check: Verified using the MATLAB `isstable()` command.

* RESULT:

- All closed-loop poles lie within the unit circle.
- System Status: Stable.



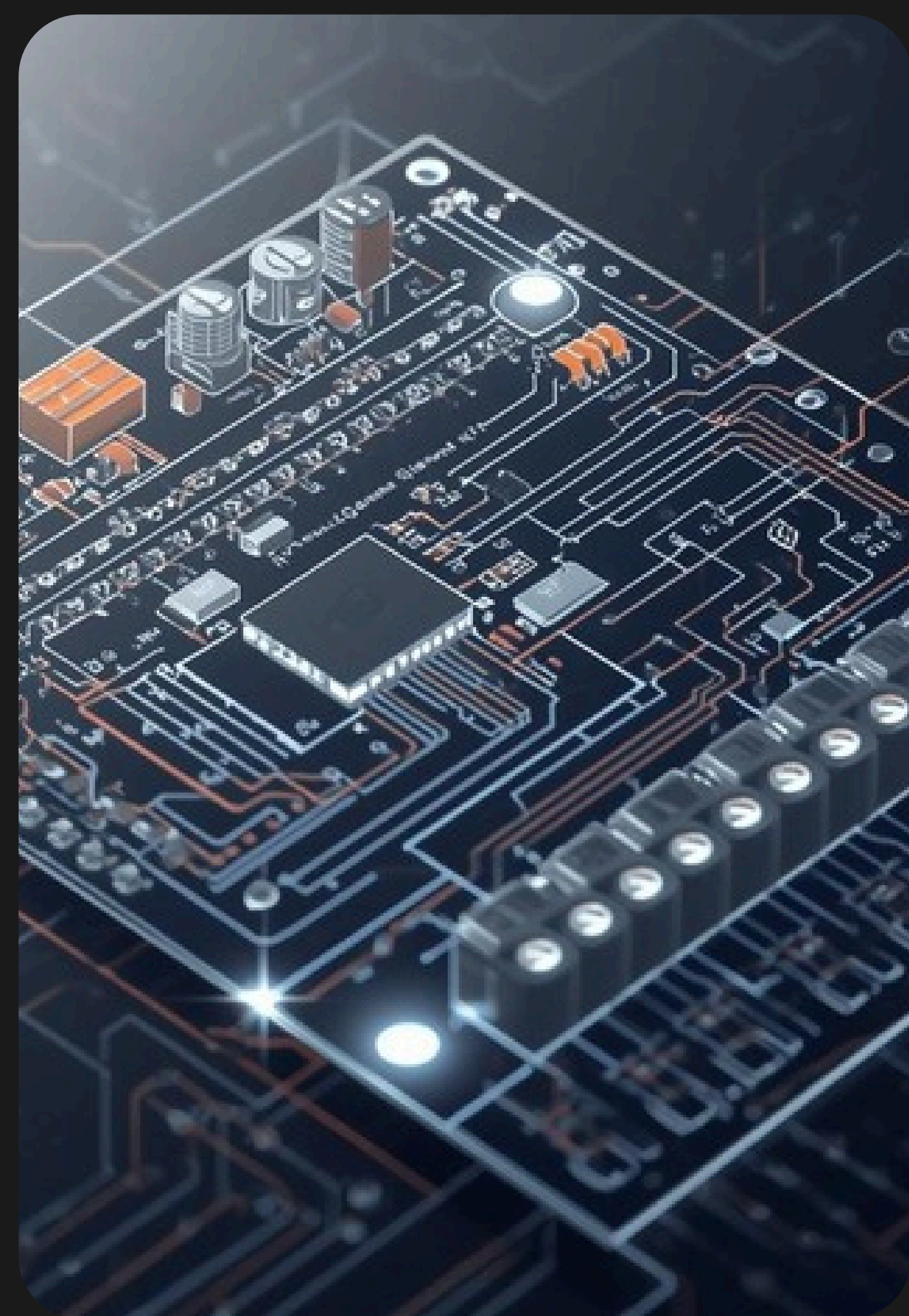
Transient Response Results (Post-Tuning)

* PERFORMANCE METRICS (STEP RESPONSE):

- Rise Time: 0.9900 seconds (Fast response to error)
- Settling Time: 1.5100 seconds
- Overshoot: 1.4725% (Very low oscillation)
- Peak Time: 2.5800 seconds

* CONCLUSION:

- The low overshoot ($<2\%$) and quick settling time confirm successful tuning.



Arduino Implementation

* CONTROL LOGIC:

Read Sensors:

- 4 IR sensors determine the current position relative to the line.

Calculate Error:

- Compute error = Setpoint - Current_Position.

PID Algorithm:

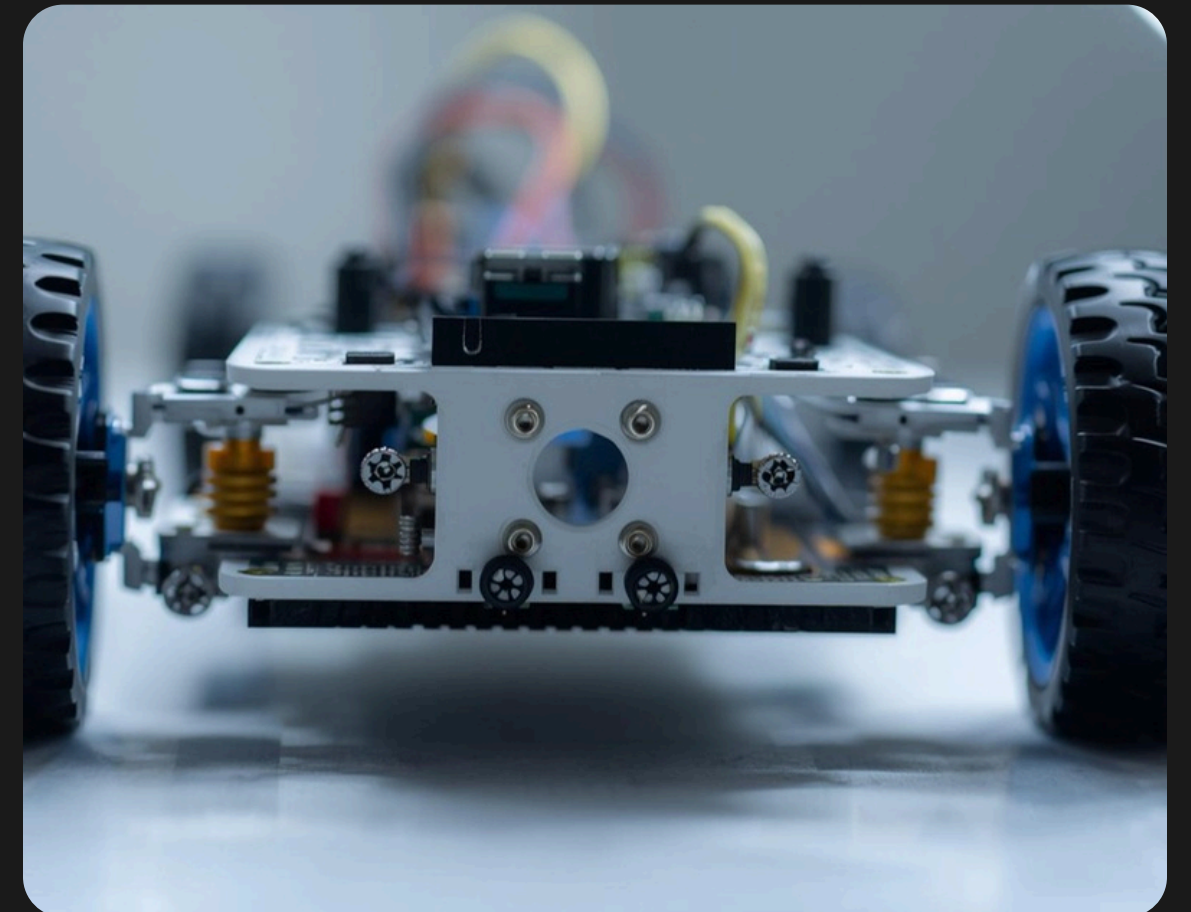
- $P = \text{error}$
- $I = I + \text{error}$
- $D = \text{error} - \text{lastError}$
- $\text{PID_value} = (K_p * P) + (K_i * I) + (K_d * D)$

* ACTUATION:

Send PWM signals via L298N driver

* MOTOR OUTPUT:

- $\text{LeftSpeed} = \text{BaseSpeed} + \text{PID_value}$
- $\text{RightSpeed} = \text{BaseSpeed} - \text{PID_value}$



Real-Time Monitoring

* **INTERFACE:** MATLAB GUI.

* **COMMUNICATION:**

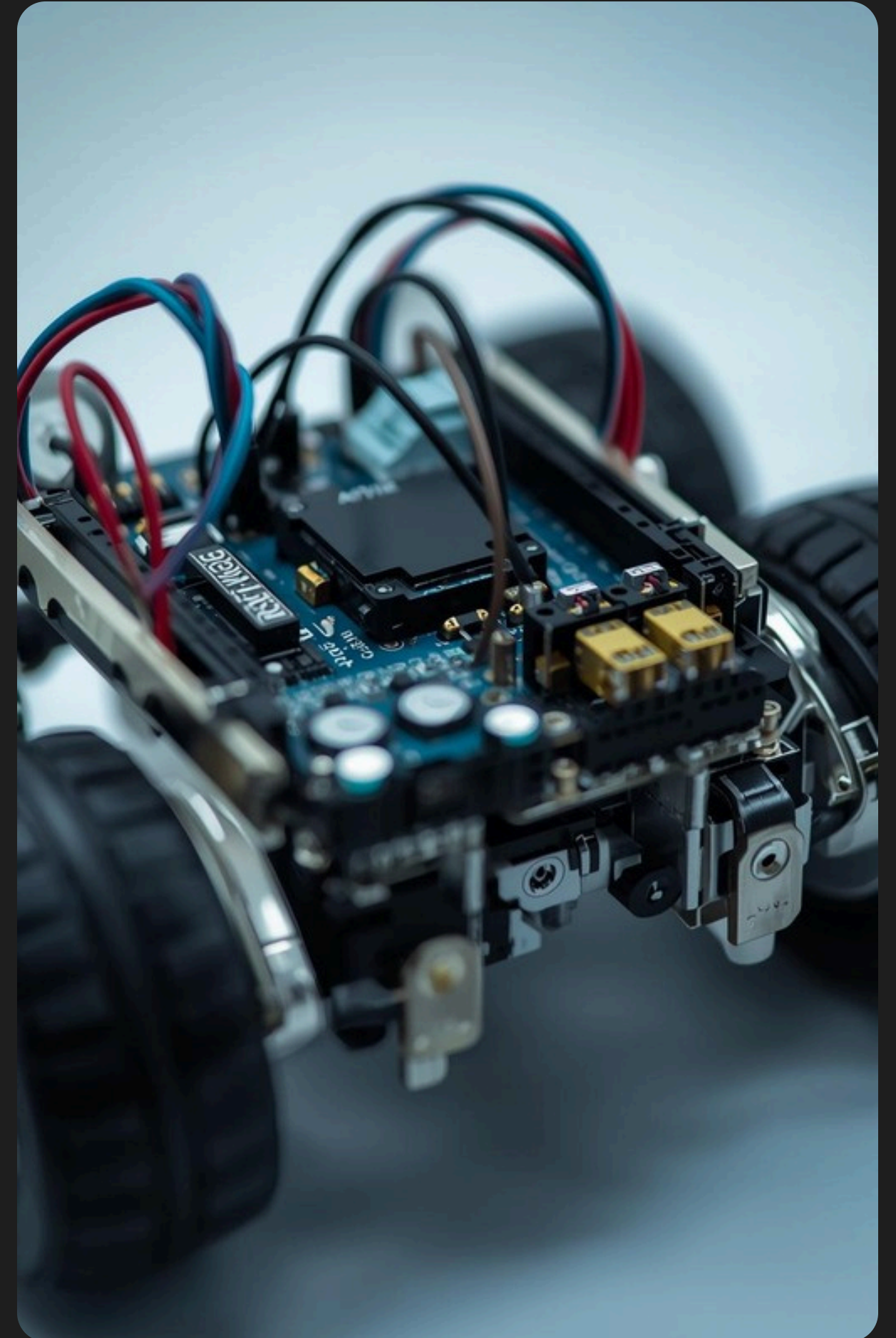
Serial connection established between Arduino and MATLAB.

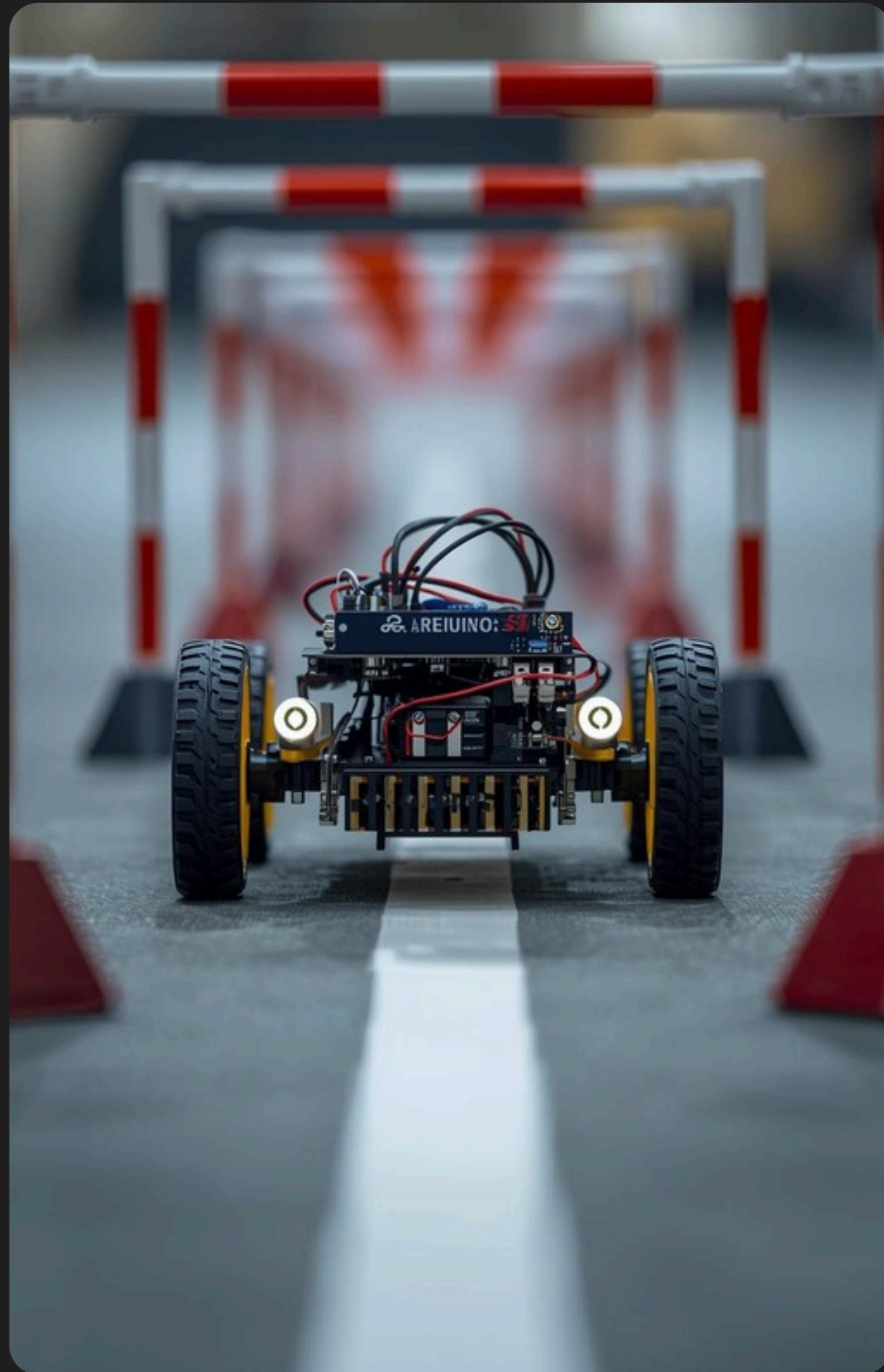
* **FUNCTIONALITY:**

- User sets the reference setpoint.
- MATLAB receives live error data from the robot.

Visualization:

- Real-time plotting of the tracking error allows for immediate performance verification.





Challenges & Solutions

* SENSOR NOISE:

- Challenge: IR sensors fluctuating due to ambient light or surface texture.
- Solution: Ensured consistent lighting and calibrated thresholds.

* FRICTION & SLIPPAGE:

- Challenge: Non-linear physical factors affecting the transfer function model.
- Solution: Robust PID tuning (Integral term) helped correct steady-state errors caused by friction.

* POWER DROP:

- Challenge: Battery voltage dropping affected motor speed.
-
- Solution: Used Li-ion batteries for consistent discharge rates.

Thank You

